2020-06-09: More groups were enabled and the total enabled groups is about 20%.

- 2020-06-10: More groups were enabled and the total enabled groups is about 21%.

- 2020-06-14: More groups were enabled and the total enabled groups is about 22%. More Sidekiq workers were added and the impact on overall GitLab.com was closely monitored during this rollout. The conclusion was made that it's safe to double the number of Sidekiq workers for Elasticsearch jobs.

- 2020-06-15: More groups were enabled and the total enabled groups is about 25%.

- 2020-06-15: More groups were enabled and the total enabled groups is about 26%. Number of Sidekiq workers were doubled again and monitoring data showed it's safe to keep the number of Sidekiq workers used in this rollout. The increased processing power would allow much faster rollout.

- 2020-06-16: More groups were enabled and the total enabled groups is about 27%.

- 2020-06-16: More groups were enabled and the total enabled groups is about 39%.

- 2020-06-17: More groups were enabled and the total enabled groups is about 64%.

- 2020-06-23: More groups were enabled and the total enabled groups is about 75%. However, a High CPU utilization issue was encountered in the Elasticsearch cluster during this batch enablement. A problemetic regex in the code analyzer was believed to be the culprit.

- 2020-06-24: the problemetic regex was fixed

- 2020-06-29: Following the code analyzer regex fix, another Elasticsearch cluster re-indexing was done successfully.

- 2020-07-02: A new feature was added to ensure newly created paid groups to be indexed.

- 2020-07-06: More groups were enabled and the total enabled groups is about 87%.

- 2020-07-08: More groups were enabled and the total enabled groups is about 93%.

- 2020-07-09: More groups were enabled and the total enabled groups is about 98%.

- 2020-07-10: All the paid groups are enabled on GitLab.com!

SECTION: engineering/ai/search/jtbd.md

Overview

The goal of this page is to create, share and iterate on the Jobs to be Done (JTBD) and their corresponding job statements for the Global Search group. Our goal is to utilize the JTBD framework to better understand our buyers' and users' needs.

Goals

Utilize JTBD and job statements to:

- Understand our users' motivations.
- Validate identified use cases and solutions.
- Continuously test and iterate features to ensure we are meeting our customers' and users' needs.
- Create a transparent view for our stakeholders into the current and future state of the product.

Jobs To Be Done

{{% product/jtbd "Global Search" %}}

SECTION: engineering/architecture/_index.md

Complexity at Scale

As GitLab grows, through the introduction of new features and improvements on existing ones, so does its complexity. This effect is compounded by the care and feeding of a single codebase that supports the wide variety of environments in which it runs, from small self-managed instances to large installations such as GitLab.com. The company itself adds to this complexity from an organizational perspective: hundreds employees worldwide contribute in one way or another to both the product and the company, using GitLab.com on a daily basis to do their job. Teams members in Engineering are directly responsible for the codebase and its operation, for the infrastructure powering GitLab.com, and for the support of customers running self-managed instances. Likewise, team members in the Product organization chart the future of the product.

Our values, coupled with strong know-how and unparalleled dedication, provide critical guidance to manage these complexities. At the same time, we have known for some time we are crossing a threshold in which complexity at scale, both technical and organizational, is playing a significant role. We know we need to fine-tune both our technical discipline (so we can integrate it across the organization) and our organizational amplification (so we can span and leverage the entire organization) to ensure we can continue to deliver on our values. In this context, we have been exploring the adoption of Architecture or Engineering Practice to help us in this regard.

Architecture

Martin Fowler's Software Architecture Guide provides an excellent discussion on the notion of Architecture, and there is much to be learned from this and other sources. The question before us is,

then, how to contextualize those learnings and apply them at GitLab.

Much like the rest of the software world, we have been wary of all the negative baggage that Architecture implies, particularly as some of that baggage would seemingly fly in the face of our values. This is why we have taken the time to carefully consider what Architecture means for us, and how to implement it in alignment with our values and at the scale that both the product and the company demand.

At GitLab, Architecture is not a dedicated role
(i.e., no such title exists in the company). We understand Architecture as a component of all technical roles, a set of practices to leverage the vast amount of experience distributed across the company, and a workflow to ensure we can continue to scale efficiently.

Architecture at GitLab

At GitLab, Architecture is a collaborative process. It is also:

- A collection of practices that provide technical frameworks to guide (rather than dictate) our thinking, design, and discussions so we can iterate quickly and deliver results. These include the Scalability Practice. Others are in the works (such as the Availability Practice).
- A collaborative workflow that provides the necessary organizational solution to foster inclusion, and drive ideas and priorities from all corners of the company.
- A collection of design documents and roadmaps which are artifacts resulting from the Architecture Design Workflow.

Such definition implies a solid reliance on collaboration rather than authority to efficiently and transparently drive decisions, engage stakeholders, and promote trust across the organization

Artifacts: roadmaps and design documents

We strive for results and concrete outcomes, which in this case entail roadmaps and design documents. Roadmaps are documents that aggregate many Design documents.

Architecture as a practice is everyone's responsibility

Architecture at GitLab is not a dedicated role but it is notably engrained in senior technical leadership roles, where the roles' levels and their sphere of influence determine responsibilities within the practice.

Architecture Design Workflow

The Architecture Design Workflow is used to create design
documents that are being used to align team members across multiple iterations.

Roadmap

Following our Transparency
value, our architecture roadmap and
design documents are public.

SECTION: engineering/architecture/roadmap.md

As GitLab continues to grow and mature, it is approaching a pivotal point in which faster growth across multiple large sites and an emphasis on the enterprise will become the main challenges to contend with over the next 12 months. All within the context of a relentless focus on security, availability, and performance.

In order to meet these challenges, we need to attain extremely high levels of predictability when operating the environments. This entails stricter standardization while providing the required flexibility to support both self-managed instances and GitLab sites, which enables the use of advanced automation by leveraging advanced tooling. But even with a high confidence level on predictability, risk is a factor we must manage: things will inevitably go wrong when effecting change in the environments. Thus, we must be able to quantify acceptable risk, to react fast in the face of failure to restore stability, and to so as a single Engineering unit.

As a company, GitLab advocates cloud native as the future of software development: customers benefit from GitLab as a complete DevOps platform delivered as a single application (from issue tracking and source code management to CI/CD and monitoring, with a built-in container registry and Kubernetes integration). We must fully embrace this model ourselves. In this context then, our most critical technical strategic investment will be our own full transition to Cloud Native on stateless components, which will afford us the ability to operate and fine-tune a variety of configurations at scale while innovating feature-wise and being frugal. In tandem, we must also develop enterprise-level capabilities to meet stringent requirements imposed by said customers.

Multi-Large Sites: Cloud Native

Cloud native is an approach to application development and operation built around cloud computing, shifting the focus from individual machines to services that rely on on-demand cloud resources to deliver high service levels while adapting to a never-ending stream of changes. It relies on technologies such as containers and strategies such a microservices to deploy at high frequencies in a consistent manner. This is reflected in our beliefs on the Biggest Tailwinds, and enables a high degree of predictability, thus allowing us to better manage said service levels.

Today, GitLab consists of a monolithic GitLab Rails codebase and several supporting services (Gitaly, GitLab Workhorse, Registry, CI runners, GitLab Shell, and GitLab Pages, etc.). GitLab has therefore already adopted some facets of cloud native, but we need to move closer to this model to succeed.

However, we cannot do a wholesale switch to cloud native: we must evolve a single codebase that supports the wide variety of environments in which it runs along two parallel but distinct

paths, scaling across multiple large sites while supporting self-managed instances. Thus, even as we embark on a fuller embrace of cloud native, we must find a balance that spans both paths, allowing for the necessary transitions from one to the other. This balance requires us to continuously evaluate our needs and identify opportunities to extract specific workloads from the monolith where appropriate without diving into the trap of creating an unmanageable set of runaway microservices while re-thinking how some of the supporting services scale in this context.

Compartmentalizing State

In order to adopt container technologies, we must remove shared local state from all possible application components that do not require it, especially those using shared POSIX storage (NFS). Build logs and GitLab Pages are the two primary users of NFS in the environment, a dependency that must be eliminated.

While we will not be migrating stateful services to cloud native at this time, we need to align them with cloud native capabilities. In particular, the main database (Postgres) needs to support cloud native strategies in the application. To that end, we must implement near-zero downtime Postgres upgrades (i.e., upgrades that, at most, simply require a database failover to take effect), which will be done through Database Logical Replication, and provide testing environments with production-like data and traffic to validate database changes st scale.

A key aspect of adopting cloud native relies on the Registry, which is currently undergoing some massive changes, starting with a tighter dependency on database resources. Without the Registry, cloud native is non-functional.

Managing Risk

Change is the main channel for the introduction of risk in the environment. We have made tremendous progress in CI/CD, where the frequency of deployments is now measured in hours. With higher deployment velocity, we need the tooling to carefully manage risk post-deployment. Features flags play a key tool to ship changes early, and safely rollout them to wide audience, ensuring that feature is both stable and performant. GitLab already uses feature flags extensively, but their current implementation needs to scale to enable their full potential is helping manage risk. We also need to be able to perform rollbacks of non-recoverable failed changes.

Finally, we must be able to quantify risk, and, in doing so, further quantify how much of it we are willing to accept. Error budgets provide a framework to do this, and an organizational adherence to error budgets enables us to think about the steps we need to take organizationally to handle what happens after error budgets have been depleted. Thus, we must deliver on a full technical and organizational implementation of error budgets.

The Enterprise

Disaster Recovery

As we move deeper into the enterprise, especially within the context of multi-large sites, disaster recovery becomes a critical capability that moves from a purely technical requirement into a critical business concern. Disaster Recovery is managed through Geo on self-managed

instances, but GitLab.com has presented a particular challenge at scale, both in terms of size and growth. Additionally, GitLab.com is a public instance with a wide variety of users. Geo operates at the instance level, but this may not be appropriate for GitLab.com. We should determine what our disaster recovery requirements are for GitLab.com (recovery times, user types), and then work towards enabling Geo to handle this unique situation (which will, over time, affect other multi-large sites). Disaster recovery working group is tasked with answering the aforementioned questions.

Repository Storage

Much work has taken place to strengthen repository storage, and we must take this work to completion. In particular, Gitaly clustering is key in managing storage tiers to provide the right level of availability, durability and performance required to meet customer demands.

Cost Management

As DevOps streamlines application development, deployment and management, cost becomes a concern the application must help with. We must pivot from reactive to proactive cost management: as the application gains new capabilities and scale, its cost footprint also increases, and these cannot be managed entirely on a reactive basis outside the application. The application itself must gain a sense of awareness to aid in cost management. This awareness comes primarily through the integration with underlying infrastructure technologies, which is well-aligned with cloud native strategies: if the application is managing how much infrastructure it needs to use to deliver rock-solid service, it must also be able to provide the necessary metadata to tag resources and make decisions to optimize and manage costs.

Roadmap

The single list of all the architectural blueprints being worked on and in progress can be found in the architecture tasks issue board.

SECTION: engineering/architecture/design-documents/_index.md

Design documents are the primary artifact that the architecture design workflow revolves around. A design document describes a technical vision and a set of principles that will guide feature implementation, as we move forward. It acts as guardrails to keep team aligned.

They are version controlled documents that are constantly updated with new insights and knowledge,
after every iteration, to become even more useful with time.

Contributing

At GitLab, everyone can contribute, including to our design documents. If you would like to contribute to any of these documents, feel free to:

1. Go to the source files in the repository and select the design document you wish to

contribute to.
1. Create a merge request.
1. @ message both an author and a coach assigned to the design document, as listed below.

{{< engineering/design-documents-list folder="handbook/engineering/architecture/design-documents" >}}

SECTION: engineering/architecture/design-documents/_template.md

<!--
Before you start:

- Copy this file to a sub-directory and call it index.md for it to appear in
  the design documents list.
- Remove comment blocks for sections you've filled in.
  When your document ready for review, all of these comment blocks should be
  removed.

To get started with a document you can use this template to inform you about what you may want to document in it at the beginning. This content will change / evolve as you move forward with the proposal. You are not constrained by the content in this template. If you have a good idea about what should be in your document, you can ignore the template, but if you don't know yet what should be in it, this template might be handy.

- Fill out this file as best you can. At minimum, you should fill in the
  "Summary", and "Motivation" sections. These can be brief and may be a copy
  of issue or epic descriptions if the initiative is already on Product's
  roadmap.
- Create a MR for this document. Assign it to an Architecture Evolution
  Coach (i.e. a Principal+ engineer).
- Merge early and iterate. Avoid getting hung up on specific details and
  instead aim to get the goals of the document clarified and merged quickly.
  The best way to do this is to just start with the high-level sections and fill
  out details incrementally in subsequent MRs.

Just because a document is merged does not mean it is complete or approved. Any document is a working document and subject to change at any time.

When editing documents, aim for tightly-scoped, single-topic MRs to keep discussions focused. If you disagree with what is already in a document, open a new MR with suggested changes.

If there are new details that belong in the document, edit the document. Once a feature has become "implemented", major changes should get new blueprints.

The canonical place for the latest set of instructions (and the likely source of this file) is

content/handbook/engineering/architecture/design-documents/template.md.

Document statuses you can use:

- "proposed"
- "accepted"
- "ongoing"
- "implemented"
- "postponed"
- "rejected"

-->

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

<!--
Don't add a h1 headline. It'll be added automatically from the title front matter attribute.

For long pages, consider creating a table of contents.
-->

Summary

<!--
This section is very important, because very often it is the only section that
will be read by team members. We sometimes call it an "Executive summary",
because executives usually don't have time to read entire documents like this.
Focus on writing this section in a way that anyone can understand what it says,
the audience here is everyone: executives, product managers, engineers, wider
community members.

A good summary is probably at least a paragraph in length.
-->

Motivation

<!--
This section is for explicitly listing the motivation, goals and non-goals of
this document. Describe why the change is important, all the opportunities,
and the benefits to users.

The motivation section can optionally provide links to issues that demonstrate
interest in a document within the wider GitLab community. Links to
documentation for competing products and services is also encouraged in cases
where they demonstrate clear gaps in the functionality GitLab provides.

For concrete proposals we recommend laying out goals and non-goals explicitly,
but this section may be framed in terms of problem statements, challenges, or
opportunities. The latter may be a more suitable framework in cases where the
problem is not well-defined or design details not yet established.
-->

Goals

<!--
List the specific goals / opportunities of the document.

- What is it trying to achieve?
- How will we know that this has succeeded?
- What are other less tangible opportunities here?
-->

Non-Goals

<!--
Listing non-goals helps to focus discussion and make progress. This section is
optional.

- What is out of scope for this document?
-->

Proposal

<!--
This is where we get down to the specifics of what the proposal actually is,
but keep it simple! This should have enough detail that reviewers can
understand exactly what you're proposing, but should not include things like
API designs or implementation. The "Design Details" section below is for the
real nitty-gritty.

You might want to consider including the pros and cons of the proposed solution so that they can be
compared with the pros and cons of alternatives.
-->

Design and implementation details

<!--
This section should contain enough information that the specifics of your
change are understandable. This may include API specs (though not always
required) or even code snippets. If there's any ambiguity about HOW your
proposal will be implemented, this is the place to discuss them.

If you are not sure how many implementation details you should include in the
document, the rule of thumb here is to provide enough context for people to
understand the proposal. As you move forward with the implementation, you may

need to add more implementation details to the document, as those may become valuable context for important technical decisions made along the way. A document is also a register of such technical decisions. If a technical decision requires additional context before it can be made, you probably should document this context in a document. If it is a small technical decision that can be made in a merge request by an author and a maintainer, you probably do not need to document it here. The impact a technical decision will have is another helpful information - if a technical decision is very impactful, documenting it, along with associated implementation details, is advisable.

If it's helpful to include workflow diagrams or any other related images. Diagrams authored in GitLab flavored markdown are preferred. In cases where that is not feasible, images should be placed under images/ in the same directory as the index.md for the proposal.
-->

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered, although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

SECTION: engineering/architecture/design-documents/activity_pub/_index.md

{{< engineering/design-document-header >}}

Summary

The end goal of this proposal is to build interoperability features into GitLab so that it's possible on one instance of GitLab to open a merge request to a project hosted on an other instance, merging all willing instances in a global network.

To achieve that, we propose to use ActivityPub, the w3c standard used by the Fediverse. This will allow us to build upon a robust and battle-tested protocol, and it will open GitLab to a wider community.

Before starting implementing cross-instance merge requests, we want to start with smaller steps, helping us to build up domain knowledge about ActivityPub and creating the underlying architecture that will support the more advanced features. For that reason, we propose to start with implementing social features, allowing people on the Fediverse to subscribe to activities on GitLab, for example to be notified on their social network of choice when their favorite project hosted on GitLab makes a new release.

As a bonus, this is an opportunity to make GitLab more social and grow its audience.

Description of the related tech and terms

Feel free to jump to Motivation if you already know what ActivityPub and the Fediverse are.

Among the push for decentralization of the web, several projects tried different protocols with different ideals behind their reasoning. Some examples:

- Secure Scuttlebutt (or SSB for short)
- Dat
- IPFS,
- Solid

One gained traction recently: ActivityPub, better known for the colloquial Fediverse built on top of it, through applications like Mastodon (which could be described as some sort of decentralized Facebook) or Lemmy (which could be described as some sort of decentralized Reddit).

ActivityPub has several advantages that makes it attractive to implementers and could explain its current success:

- It's built on top of HTTP. You don't need to install new software or to tinker with TCP/UDP to implement ActivityPub, if you have a webserver or an application that provides an HTTP API (like a rails application), you already have everything you need.
- It's built on top of JSON. All communications are basically JSON objects, which web developers are already used to, which simplifies adoption.
- It's a W3C standard and already has multiple implementations. Being piloted by the W3C is a guarantee of stability and quality work. They have profusely demonstrated in the past through their work on HTML, CSS or other web standards that we can build on top of their work without the fear of it becoming deprecated or irrelevant after a few years.

The Fediverse

The core idea behind Mastodon and Lemmy is called the Fediverse. Rather than full decentralization, those applications rely on federation, in the sense that there still are servers and clients. It's not P2P like SSB, Dat and IPFS, but instead a galaxy of servers chatting with each other instead of having central servers controlled by a single entity.

The user signs up to one of those servers (called instances), and they can then interact with users either on this instance, or on other ones.

From the perspective of the user, they access a global network, and not only their instance. They see the articles posted on other instances, they can comment on them, upvote them, etc.

What happens behind the scenes:
their instance knows where the user they reply to is hosted. It contacts that other instance to let them know there is a message for them - somewhat similar to SMTP. Similarly, when a user subscribes to a feed, their instance informs the instance where the feed is hosted of this subscription. That target instance then posts back messages when new activities are created. This allows for a push model, rather than a constant poll model like RSS. Of course, what was just described is the happy path; there is moderation, validation and fault tolerance happening all the way.

ActivityPub

Behind the Fediverse is the ActivityPub protocol. It's a HTTP API attempting to be as general a social network implementation as possible, while giving options to be extendable.

The basic idea is that an actor sends and receives activities. Activities are structured JSON messages with well-defined properties, but are extensible to cover any need. An actor is defined by four endpoints, which are contacted with the application/ld+json; profile="https://www.w3.org/ns/activitystreams" HTTP Accept header:

- GET /inbox: used by the actor to find new activities intended for them.
- POST /inbox: used by instances to push new activities intended for the actor.
- GET /outbox: used by anyone to read the activities created by the actor.
- POST /outbox: used by the actor to publish new activities.

Among those, Mastodon and Lemmy only use POST /inbox and GET /outbox, which are the minimum needed to implement federation:

- Instances push new activities for the actor on the inbox.
- Reading the outbox allows reading the feed of an actor.

Additionally, Mastodon and Lemmy implement a GET / endpoint (with the mentioned Accept header). This endpoint responds with general information about the actor, like name and URL of the inbox and outbox. While not required by the standard, it makes discovery easier.

While a person is the main use case for an actor, an actor does not necessarily map to a person. Anything can be an actor: a topic, a subreddit, a group, an event. For GitLab, anything with activities (in the sense of what GitLab means by "activity") can be an ActivityPub actor. This includes items like projects, groups, and releases. In those more abstract examples, an actor can be thought of as an actionable feed.

ActivityPub by itself does not cover everything that is needed to implement the Fediverse. Most notably, these are left for the implementers to figure out:

- Finding a way to deal with spam. Spam is handled by authorizing or blocking ("defederating") other instances.
- Discovering new instances.
- Performing network-wide searches.

Motivation

Why would a social media protocol be useful for GitLab? People want a single, global GitLab network to interact between various projects, without having to register on each of their hosts.

Several very popular discussions around this have already happened:

- Share events externally via ActivityPub
- Implement cross-server (federated) merge requests
- Distributed merge requests.

The ideal workflow would be:

1. Alice registers to her favorite GitLab instance, like gitlab.example.org.
1. She looks for a project on a given topic, and sees Bob's project, even though Bob is on gitlab.com.
1. Alice selects Fork, and the gitlab.com/Bob/project.git is forked to gitlab.example.org/Alice/project.git.
1. She makes her edits, and opens a merge request, which appears in Bob's project on gitlab.com.
1. Alice and Bob discuss the merge request, each one from their own GitLab instance.
1. Bob can send additional commits, which are picked up by Alice's instance.
1. When Bob accepts the merge request, his instance picks up the code from Alice's instance.

In this process, ActivityPub would help in:

- Letting Bob know a fork happened.
- Sending the merge request to Bob.
- Enabling Alice and Bob to discuss the merge request.
- Letting Alice know the code was merged.

It does not help in these cases, which need specific implementations:

- Implementing a network-wide search.
- Implementing cross-instance forks. (Not needed, thanks to Git.)

Why use ActivityPub here rather than implementing cross-instance merge requests in a custom way? Two reasons:

1. Building on top of a standard helps reach beyond GitLab.
   While the workflow presented above only mentions GitLab, building on top
   of a W3C standard means other forges can follow GitLab
   there, and build a massive Fediverse of code sharing.
1. An opportunity to make GitLab more social. To prepare the
   architecture for the workflow above, smaller steps can be taken, allowing
   people to subscribe to activity feeds from their Fediverse social
   network. Anything that has a RSS feed could become an ActivityPub feed.
   People on Mastodon could follow their favorite developer, project, or topic
   from GitLab and see the news in their feed on Mastodon, hopefully raising
   engagement with GitLab.

Goals

- allowing to share interesting events on ActivityPub based social media
- allowing to open an issue and discuss it from one instance to an other
- allowing to fork a project from one instance to an other
- allowing to open a merge request, discuss it and merge it from one instance to an other
- allowing to perform a network wide search?

Non-Goals

- federation of private resources
- allowing to perform a network wide search?

Proposal

The idea of this implementation path is not to take the fastest route to
the feature with the most value added (cross-instance merge requests), but
to go on with the smallest useful step at each iteration, making sure each step
brings something immediately useful.

1. Implement ActivityPub for social following.
   After this, the Fediverse can follow activities on GitLab instances.
   1. ActivityPub to subscribe to project releases.
   1. ActivityPub to subscribe to project creation in topics.
   1. ActivityPub to subscribe to project activities.
   1. ActivityPub to subscribe to group activities.
   1. ActivityPub to subscribe to user activities.
1. Implement cross-instance forks to enable forking a project from an other instance.
1. Implement ActivityPub for cross-instance discussions to enable discussing
   issues and merge requests from another instance:
   1. In issues.
   1. In merge requests.
1. Implement ActivityPub to submit cross-instance merge requests to enable
   submitting merge requests to other instances.
1. Implement cross-instance search to enable discovering projects on other instances.

It's open to discussion if this last step should be included at all.
Currently, in most Fediverse apps, when you want to display a resource from

an instance that your instance does not know about (typically a user you want to follow), you paste the URL of the resource in the search box of your instance, and it fetches and displays the remote resource, now actionable from your instance. We plan to do that at first.

The question is : do we keep it at that? This UX has severe frictions, especially for users not used to Fediverse UX patterns (which is probably most GitLab users). On the other hand, distributed search is a subject complicated enough to deserve its own blueprint (although it's not as complicated as it used to be, now that decentralization protocols and applications worked on it for a while).

Design and implementation details

First, it's a good idea to get familiar with the specifications of the three standards we're going to use:

- ActivityPub defines the HTTP requests happening to implement federation.
- ActivityStreams defines the format of the JSON messages exchanged by the users of the protocol.
- Activity Vocabulary defines the various messages recognized by default.

Feel free to ping @oelmekki if you have questions or find the documents too dense to follow.

Production readiness

TBC

The social following part

This part is laying the ground work allowing to add new ActivityPub actors to GitLab.

There are 5 actors we want to implement:

- the releases actor, to be notified when given project makes a new release
- the topic actor, to be notified when a new project is added to a topic
- the project actor, regarding all activities from a project
- the group actor, regarding all activities from a group
- the user actor, regarding all activities from a user

We're only dealing with public resources for now. Allowing federation of private resources is a tricky subject that will be solved later, if it's possible at all.

Endpoints

Each actor needs 3 endpoints:

- the profile endpoint, containing basic info, like name, description, but also including links to the inbox and outbox
- the outbox endpoint, allowing to show previous activities for an actor
- the inbox endpoint, on which to post to submit follow and unfollow requests (among other things we won't use for now).

The controllers providing those endpoints are in app/controllers/activitypub/. It's been decided to use this namespace to avoid mixing the ActivityPub JSON responses with the ones meant for the frontend, and also because we may need further namespacing later, as the way we format activities may be different for one Fediverse app, for an other, and for our later cross-instance features. Also, this namespace allow us to easily toggle what we need on all endpoints, like making sure no private project can be accessed.

Serializers

The serializers in app/serializers/activitypub/ are the meat of our implementation, are they provide the ActivityStreams objects. The abstract class ActivityPub::ActivityStreamsSerializer does all the heavy lifting of validating developer provided data, setting up the common fields and providing pagination.

That pagination part is done through Gitlab::Serializer::Pagination, which uses offset pagination.
We need to allow it to do keyset pagination.

Subscription

Subscription to a resource is done by posting a
Follow activity
to the actor inbox. When receiving a Follow activity,
we should generate an Accept or Reject activity in return,
sent to the subscriber's inbox.

The general workflow of the implementation is as following:

- A POST request is made to the inbox endpoint, with the Follow activity encoded as JSON
- if the activity received is not of a supported type (e.g. someone tries to comment on the activity), we ignore it ; otherwise:
- we create an ActivityPub::Subscription with the profile URL of the subscriber
- we queue a job to resolve the subscriber's inbox URL
  - in which we perform a HTTP request to the subscriber profile to find their inbox URL (and the shared inbox URL if any)

  - we store that URL in the subscription record
- we queue a job to accept the subscription
  - in which we perform a HTTP request to the subscriber inbox to post an Accept activity
  - we update the state of the subscription to :accepted

ActivityPub::Subscription is a new abstract model, from which inherit models related to our actors, each with their own table:

- ActivityPub::ReleasesSubscription, table activitypubreleasessubscriptions
- ActivityPub::TopicSubscription, table activitypubtopicsubscriptions
- ActivityPub::ProjectSubscription, table activitypubprojectsubscriptions
- ActivityPub::GroupSubscription, table activitypubgroupsubscriptions
- ActivityPub::UserSubscription, table activitypubusersubscriptions

The reason to go with a multiple models rather than, say, a simpler actor enum in the Subscription model with a single table is because we needs specific associations and validations for each (an ActivityPub::ProjectSubscription belongs to a Project, an ActivityPub::UserSubscription does not). It also gives us more room for extensibility in the future.

Unfollow

When receiving
an Undo activity
mentioning previous Follow, we remove the subscription from our database.

We are not required to send back any activity, so we don't need any worker here, we can directly remove the record from database.

Sending activities out

When specific events (which ones?) happen related to our actors, we should queue events to issue activities on the subscribers inboxes (the activities are the same than we display in the actor's outbox).

We're supposed to deduplicate the subscriber list to make sure we don't send an activity twice to the same person - although it's probably better handled by a uniqueness validation from the model when receiving the Follow activity.

More importantly, we should group requests for a same host : if ten users are all on https://mastodon.social/, we should issue a single request on the shared inbox provided, adding all the users as recipients, rather than sending one request per user.

Webfinger

Webfinger

Mastodon
requires instance to implement the Webfinger protocol.
This protocol is about adding an endpoint at a well known location which allows to query for a resource name and have it mapped to whatever URL we want (so basically, it's used for discovery). Mastodon uses this to query other fediverse apps for actor names, in order to find their profile URLs.

Actually, GitLab already implements the Webfinger protocol endpoint through Doorkeeper
(this is the action that maps to its route),
implemented in GitLab
in JwksController.

There is no incompatibility here, we can just extend this controller. Although, we'll probably have to rename it, as it won't be related to Jwks alone anymore.

One difficulty we may have is that contrary to Mastodon, we don't only deal with users. So we need to figure something to differentiate asking for a user from asking for a project, for example. One obvious way would be to use a prefix, like user-<username>, project-<projectname>, etc. I'm pondering that from afar, while we haven't implemented much code in the epic and I haven't dig deep into Webfinger's specs, this remark may be deprecated when we reach actual implementation.

HTTP signatures

HTTP signatures

Mastodon
requires HTTP signatures,
which is yet an other standard, in order to make sure no spammer tries to impersonate a given server.

This is asymmetrical cryptography, with a private key and a public key, like SSH or PGP. We will need to implement both signing requests, and verifying them. This will be of considerable help when we'll want to have various GitLab instances communicate later in the epic.

Host allowlist and denylist

To give GitLab instance owners control over potential spam, we need to allow to maintain two mutually exclusive lists of hosts:

- the allowlist : only hosts mentioned in this list can be federated with.
- the denylist : all hosts can be federated with but the ones mentioned in that list.

A setting should allow the owner to switch between the allowlist and the denylist.
In the beginning, this can be managed in rails console, but it will

ultimately need a section in the admin interface.

Limits and rollout

In order to control the load when releasing the feature in the first
months, we're going to set gitlab.com to use the allowlist and rollout
federation to a few Fediverse servers at a time, so that we can see how it
takes the load progressively, before ultimately switching to denylist
(note: there are
some ongoing discussions
regarding if federation should be activated on gitlab.com or not).

We also need to implement limits to make sure the federation is not abused:

- limit to the number of subscriptions a resource can receive.
- limit to the number of subscriptions a third party server can generate.

The cross-instance issues and merge requests part

We'll wait to be done with the social following part before designing this
part, to have ground experience with ActivityPub.

SECTION: engineering/architecture/design-documents/ai_context_abstraction_layer/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

To enhance the grounding of our AI features, there is a need for Retrieval Augmented Generation (RAG). As RAG evolves, no single method or storage solution currently addresses all potential use cases. Additionally, with advances in LLMs and larger context windows, our solution must remain adaptable. While we are already using Elasticsearch for most search features and embeddings, we still don't have all self-managed customers running Elasticsearch, so we will want to support some features with Postgres as well. Hence, creating an abstraction layer that unifies various RAG and embedding solutions will enable us to support diverse use cases efficiently.

Motivation

The RAG abstraction layer will provide several key benefits:

- Customer Flexibility: Customers that do not run Elasticsearch today will be able to get some RAG features powered by Postgres.
- Avoid Vendor Lock-in: The architecture is designed to prevent dependency on a single vendor.
- Modular Feature Development: Features can be built independently of underlying data store

constraints.
- Adaptability: Our solution can evolve as new techniques, models, and vendors emerge.

Goals

- Enable the Global Search team to support multiple vector databases and embedding models while maximising the amount of code shared across databases.
- Build an abstraction layer for diverse features like chat routing, memory, issue search, and code context.

Proposal

Here's the outline of the proposed interface for the GitLab Data Layer.

Setup

When admins set up a new supported database, they are going to add it to the GitLab Data Layer configuration. It needs to be one of the databases that has an adapter implemented.

This would be configured in the gitlab.rb file like:

```
ruby
gitlabrails['aicontextabstractionlayer'] = {
  enabled: true,
  databases: {
    es1: {
      adapter: 'elasticsearch',
      options: {
          prefix: 'gitlab',  Optional, but important to allow using the same DB for multiple
instances
          url: 'https://elastic.host'
      }
    },
    os1: {
      adapter: 'opensearch',
      options: {
          prefix: 'gitlab',  Optional, but important to allow using the same DB for multiple
instances
          url: 'https://elastic.host'
      }
    },
    pg1: {
      adapter: 'postgresql',
      options: {
          prefix: 'gitlab',  Optional, but important to allow using the same DB for multiple
instances
          host: 'postgres.host',
          username: 'postgres',
          password: '..'
```

```
      }
    },
     ...
  }
}
```

Stable interface

In this section I'll attempt to outline how users (team members) can define the collection, migrate the data, and query it.

Defining the collection

The first step will be to create a top level class for the collection.

We're going to have one class per collection that would be responsible for pointing to other reference/indexing/searching classes as well as for defining routing.

```ruby
module Ai
  module Search
    module Context
      module Collections
        class Issue
          class << self
            def referenceclass
              Issue
            end

            ...
          end
        end
      end
    end
  end
end
```

This is the reference class, which is responsible for generating the document as well as serialization/deserialization from/to the ZSETs.

```ruby
module Ai
  module Search
    module Context
      class Issue < Reference
        include Ai::Search::Context::Concerns::DatabaseReference
```

```
        MODELS = {
          0: {
              field: 'embeddingsgecko',
              model: 'textembedding-gecko@003',
              chunkingstrategy: {
                We can start with a naive approach first, but eventually
                we'll need to introduce chunking strategies
                ..
              }
            },
          1: {
              field: 'embeddingsmistral',
              model: 'mistral7B',
              chunkingstrategy: {
                We can start with a naive approach first, but eventually
                we'll need to introduce chunking strategies
                ..
              }
            }
        }

        override :asindexedjsons
        def asindexedjsons
          [
            {
              issueid: target.issueid,
              namespaceid: target.namespaceid,
              embeddinggecko: Ai::Search::Context.embedding(embeddingscontent, collection:
:issues, modelversion: 0),
              embeddingmistral: Ai::Search::Context.embedding(embeddingscontent, collection:
:issues, modelversion: 1)
               ...
            }
          ]
        end

        private

        def embeddingscontent
          "issue with title '{target.title}' and description '{target.description}'"
        end
      end
    end
  end
end
```

Data migration

ruby

```
class AddIssueCollection < Gitlab::Ai::Context::Migration[1.0]
  milestone '18.0'

  def change
    createcollection :issues, numberofpartitions: 5 do |c|
      c.bigint :issueid
      c.bigint :namespaceid
      c.bigint :projectid
      c.prefix :traversalids
      c.vector :embeddingsgecko, options: { dimensions: 768, m: 16, efconstruction: 100 }
      c.integer :embeddingsversion

       These will only be used with pgvector adapter since Elasticsearch
       indexes fields automatically
       we could also consider adding index: true to field definitions as syntactic sugar
      c.index [:namespaceid, :projectid]
      c.index :traversalids
      c.index :embeddingsgecko
    end
  end
end
```

Tracking the state of the collection

We plan to add 1 record for each collection to track some dynamic attributes like:

- searchembeddingversion: 0
- indexingembeddingversions: [0, 1]

Migrating the collection

ruby

```
class AddContentToIssueCollection < Gitlab::Ai::Context::Migration[1.0]
  milestone '18.0'

  def change
    addfield :issues, field: :content, type: :text
  end
end
```

ruby

```
class BackfillContentToIssueCollection < Gitlab::Ai::Context::Migration[1.0]
  milestone '18.0'

  def change
    backfillcollection :issues, field: :content
  end
end
```

Tracking the state of migrations

We're going to add a new table (in the main DB) to track the state of all the migrations. The table could looks something like

- version: text
- status: pending, inprogress, finished, failed (enum)
- createdat
- updatedat
- completedat
- metadata/info: jsonb

We're going to have helper methods to get the current migration state(s) since some code will need to know whether or not a specific migration has been completed.

ruby
Ai::Search::Context.migration(:backfillcontenttoissuecollection).state

or

Ai::Search::Context.migrationfinished?(:backfillcontenttoissuecollection)

Migrating between different models

We plan to make it easy to migrate between different LLMs without any downtime. The plan is to have a
list of models mapped to field names. That way we can track current/active models and switch to the new one as soon as the backfill is completed.

mermaid
sequenceDiagram
    participant Orchestrator
    participant Collections table on pg main
    participant Migrations table on pg main
    participant Embeddings Store
    participant AI Gateway
    Note over Orchestrator: Migrate embeddings from one model to another

    Orchestrator->>Collections table on pg main: add entry to list of models with corresponding fields
    Orchestrator->>Collections table on pg main: append new version to indexingembeddingversions
    Orchestrator->>Migrations table on pg main: create migration record to add field
    Migrations table on pg main->>Embeddings Store: migration to add field
    Embeddings Store->>Migrations table on pg main: mark migration as completed
    Orchestrator->>Migrations table on pg main: create migration record to backfill data
    Migrations table on pg main->>Embeddings Store: batched backfill migration

Embeddings Store->>AI Gateway: get embeddings in bulk
AI Gateway->>Embeddings Store: -
Embeddings Store->>Migrations table on pg main: mark migration as completed
Orchestrator->>Migrations table on pg main: create migration record to nullify data in old field
Migrations table on pg main->>Embeddings Store: migration to nullify data in old field
Embeddings Store->>Migrations table on pg main: mark migration as completed
Orchestrator->>Collections table on pg main: remove old version from indexingembeddingversions
Orchestrator->>Collections table on pg main: update searchembeddingversion to new version

Ingestion

ruby
Ai::Search::Context.track!(objects)

This will create references under the hood and add these to sharded redis ZSET's, which are used as deduplicated queues.

Then another worker/workers processes the queue to be within the specified rate limits.

We're going to introduce concerns to include into the model so that changes are tracked automatically. Similar to Elastic::ApplicationVersionedSearch.

Note: One indexing event might need to be distributed to multiple separate collections. For example,
it could be different chunking strategies for code or another document type.

Retrieval

ruby
embedding = Ai::Search::Context.embedding('Abstraction layer', collection: :issues, modelversion: 0, user: currentuser)
or for batching
embeddings = Ai::Search::Context.embeddings(['Abstraction layer'], collection: :issues, modelversion: 0, user: currentuser)

Ai::Search::Context.collection(:issues).query(prefix: { traversalids: '9970-' }, modelversion: 0, embeddings: embedding, limit: 5)

If we find that it is difficult to represent all our query logic in a Ruby based API, we could consider using the GLQL compiler to transform the query into an AST. Currently,
it is being rewritten in Rust,
which allows us to incorporate it as part of the abstraction layer in the monolith. We decided not to start with GLQL as it introduces new dependencies on other complex projects and this use

case is not aligned with the original design goal of GLQL to be a user facing query language to present data in the GitLab UI.

Another alternative is to use a JSON object to represent the query, similar to the approach Elasticsearch takes.
However, one downside of this approach is that it makes expressing complex AND/OR queries quite challenging.
A SQL-like syntax might be more convenient and familiar to the team members.

Important note: As part of the initial implementation we should have redaction logic built-in.

Initial features

We intend to build the abstraction layer so that it can support a broad range of features in GitLab. The following are some features we intend to support eventually:

- Documentation embeddings
- Code embeddings
- Issue/MR/Epic embeddings
- CI logs embeddings

Since code (stored in Gitaly, updated by git push, has multiple versions at any one time) is very different to issues and merge requests (stored in Postgres, updated by application code, only one version at a time) we expect the Global Search team will need to build a reference implementation for both. The architecture we build will need to be flexible to many different ways of constructing and searching embeddings but it will require experimentation to learn which approaches are useful for specific features. As such we expect the Global Search team to build an experimental reference implementation that may not be rolled out to all customers. We will make our best effort to design the feature to be useful based on what we know today but we will not delay the initial reference implementation while we wait for deeper research into this topic.

Once we have clear reference implementations in place it will be easier for other feature teams to build the indexing and search functionality they need to support their features.

SECTION: engineering/architecture/design-documents/ai_context_management/_index.md

{{< engineering/design-document-header >}}

Glossary

- AI Context. In the scope of this technical blueprint, the term "AI Context" refers to supplementary information
provided to the AI system alongside the primary prompts.
- AI Context Policy. The "AI Context Policy" is a user-defined and user-managed mechanism allowing precise
control over the content that can be sent to the AI as contextual information. In the context

of this blueprint, the
AI Context Policy is suggested as a YAML configuration file.
- AI Context Policy Management. Within this blueprint, "Management" encompasses the user-driven processes of
creating, modifying, and removing AI Context Policies according to specific requirements and preferences.
- Automatic AI Context. AI Context, retrieved automatically based on the active document. Automatic AI Contex
can be the active document's dependencies (modules, methods, etc., imported into the active document), some
search-based, or other mechanisms over which the user has limited control.
- Supplementary User Context: User-defined AI Context, such as open tabs in IDEs, local files, and folders, that the user
provides from their local environment to extend the default AI Context.
- AI Context Retriever: A backend system capable of:
  - communicating with AI Context Policy Management
  - fetching content defined in Automatic AI Context and Supplementary User Context (complete files, definitions,
 methods, etc.), based on the AI Context Policy Management
  - correctly augment the user prompt with AI Context before sending it to LLM. Presumably, this part is already
 handled by AI Gateway.
- Project Administrator. In the context of this blueprint, "Project Administrator" means any individual with the
"Edit project settings" permission ("Maintainer" or "Owner" roles, as defined in Project members permissions).

Summary

Correct context can dramatically improve the quality of AI responses. This blueprint aims to accommodate AI Context
seamlessly into our offering by architecting a solution that is ready for this additional context coming from different
AI features.

However, we recognize the importance of security and trust, which automatic solutions do not necessarily provide. To
address any concerns users might have about the content fed into the AI Context, this blueprint suggests providing them
with control and customization options. This way, users can adjust the content according to their preferences and have a
clear understanding of what information is being utilized.

This blueprint proposes a system for managing AI Context at the Project Administrator and individual
user levels. Its goal is to allow Project Administrator to set high-level rules for what content can be included as context for AI
prompts while enabling users to specify Supplementary User Context for their prompts. The

global AI Context Policy will use a YAML
configuration file format stored in the same Git repository. The suggested format of the YAML configuration files
is discussed below.

Motivation

Ensuring the AI has the correct context is crucial for generating accurate and relevant code suggestions or responses.
As the adoption of AI-assisted development grows, it's essential to give organizations and users control over what project
content is sent as context to AI models. Some files or directories may contain sensitive information that should not
be shared. At the same time, users may want to provide additional context for their prompts to get more
relevant suggestions. We need a flexible AI Context management system to handle these cases.

Goals

For Project Administrators

- Allow Project Administrators set the default AI Context Policy to control whether content can or cannot be
automatically included in the AI Context when making requests to LLMs
- Allow Project Administrators to specify exceptions to the default AI Context Policy
- Provide a UI to manage the default AI Context Policy and its exceptions list easily

For users

- Allow to set Supplementary User Context to include as AI context for their prompts
- Provide a UI to manage Supplementary User Context easily

Non-Goals

- AI Context Retriever architecture - different environments (Web, IDEs) will probably implement their retrievers.
However, the unified public interface of the retrievers should be considered.
- Extremely granular controls like allowing/excluding individual lines of code
- Storing entire file contents from user projects, only paths will be persisted

Proposal

The proposed architecture consists of 3 main parts:

- AI Context Retriever
- AI Context Policy Management
- Supplementary User Context

There are several different ongoing efforts related to various implementations of AI Context Retriever both

for Web, and for IDEs.
Because of that, the architecture for AI Context Retriever is beyond the scope of this blueprint. However, in the
context of this blueprint, it is assumed that:

- AI Context Retriever is capable of automatically retrieving and fetching Automatic AI Context and passing it
on as AI Context to LLM.
- AI Context Retriever can automatically retrieve and fetch Supplementary User Contextand pass it on as AI Context to LLM.
- AI Context Retriever implementation can ensure that any content passed as AI Context to a model
adheres to the global AI Context Policy.
- AI Context Retriever can trim the AI Context to meet the contextual window requirement for a specific LLM used for that or another Duo feature.

AI Context Policy Management proposal

To implement the AI Context Policy Management system, it is proposed to:

- Introduce the YAML file format for configuring global policies
- In the YAML configuration file, support two aicontextpolicy types:
  - block: blocks all content except for the specified exclude paths. Excluded files are allowed. (Default)
  - allow: allows all content except for the specified exclude paths. Excluded files are blocked.
  - version: specifies the schema version of the AI context file. Starting with version: 1. If omitted treated as the latest version known to the client.
- In the YAML configuration file, support glob patterns to exclude certain paths from the global policy
- Support nested AI Context Policies to provide a more granular control of AI Context in sub-folders. For
example, a policy in /src/tests would override a policy in /src, which, in its turn, would override a
global AI Context Policy in /.

Supplementary User Context proposal

To implement the Supplementary User Context system, it is proposed to:

- Introduce user-level UI to specify Supplementary User Context for prompts. A particular implementation of the UI could
differ in different environments (IDEs, Web, etc.), but the actual design of these implementations is beyond the scope of
this architecture blueprint
- The user-level UI should communicate to the user what is in the Supplementary User Context at any moment.
- The user-level UI should allow the user to edit the contents of the Supplementary User Context.

Optional steps

- Provide UI for Project Administrators to configure global AI Context Policy. Source Editor can be used as the editor for this type of YAML file format, similar to the Security Policy Editor.
- Implement a validation mechanism for AI Context Policies to somehow notify the Project Administrators in case of the invalid format of the YAML configuration file. It could be a job in CI. But to catch possible issues proactively, it is also advised to introduce the validation step as part of the pre-push static analysis

Design and implementation details

- YAML Configuration File Format: The proposed YAML configuration file format for defining the global AI Context Policy is as follows:

```
yaml
aicontextpolicy: [allow|block]

exclude:
- glob//pattern
```

The aicontextpolicy section specifies the current policy for this and all underlying folders in a repo.

The exclude section specifies the exceptions to the aicontextpolicy. Technically, it's an inversion of the policy.
For example, if we specify foobar.js in exclude:

- for the allow policy, it means that foobar.js will be blocked
- for the block policy, it means that foobar.js will be allowed

- User-Level UI for Supplementary User Context: The UI for specifying Supplementary User Context for prompts can be implemented differently depending on the environment (IDEs, Web, etc.). However, the implementation should ensure users can provide additional context for their prompts. The specified Supplementary User Context for each user can be stored as:

  - a preference stored in the user profile in GitLab

    - Pros: Consistent across devices and environments (Web, IDEs, etc.)
    - Cons: Additional work in the monolith, potentially a lot of new read/writes to a database

  - a preference stored in the local IDE/Web storage

- Pros: User-centric, local to user environment
- Cons: Different implementations for different environments (Web, IDEs, etc.), doesn't survive switching
environment or device

In both cases, the storage should allow the preference to be associated with a particular repository. Factors
like data consistency, performance, and implementation complexity should guide the decision on what type of storage
to use.

- To mitigate potential performance and scalability issues, it would make sense to keep AI Context Retriever, and
AI Context Policy Management in the same environment as the feature needing those. It would be Language Server for Duo features in IDEs and different
services in the monolith for Duo features on the Web.

Data flow

Here's the draft of the data flow demonstrating the role of AI Context using the Code Suggestions feature as an example.

mermaid
sequenceDiagram
participant CS as Code Suggestions
participant CR as AI Context Retriever
participant PM as AI Context Policy Management
participant LLM as Language Model

CS->>CR: Request Code Suggestion
CR->>CR: Retrieve Supplementary User Context list
CR->>CR: Retrieve Automatic AI Context list
CR->>PM: Check AI Context against Policy
PM-->>CR: Return valid AI Context list
CR->>CR: Fetch valid AI Context
CR->>LLM: Send prompt with final AI Context
LLM->>LLM: Generate code suggestions
LLM-->>CS: Return code suggestions
CS->>CS: Present code suggestions to the user

In case the AI Context Retriever fails to fetch any content from the AI Context, the prompt is sent with
AI Context, which was successfully fetched. In a low-probability case, when AI Context Retriever cannot fetch any content, the prompt should be sent out as-is.

Alternative solutions

JSON Configuration Files

- Pros: Widely used, easier integration with web technologies.
- Cons: Less readable compared to YAML for complex configurations.

Database-Backed Configuration

- Pros: Centralized management, dynamic updates.
- Cons: Not version controlled.

Environment Variables

- Pros: Simplifies configuration for deployment and scaling.
- Cons: Less suitable for complex configurations.

Policy as Code (without YAML)

- Pros: Better control and auditing with versioned code.
- Cons: It requires users to write code and us to invent a language for it.

Policy in .aiignore and other Git-like files

- Pros: Provides a straightforward approach, identical to the allow policy with the list of exclude suggested in this blueprint
- Cons: Supports only the allow policy; the processing of this file type still has to be implemented

Based on these alternatives, the YAML file was chosen as a format for this blueprint because of versioning
in Git, and more versatility compared to the .aiignore alternative.

Suggested iterative implementation plan

Please refer to the Proposal for a detailed explanation of the items in every iteration.

Iteration 1

- Introduce the global .ai-context-policy.yaml YAML configuration file format and schema for this file type
as part of AI Context Policy Management.
- AI Context Retrievers introduce support for Supplementary User Context.
- Optional: validation mechanism (like CI job and pre-push static analysis) for .ai-context-policy.yaml

Success criteria for the iteration: Prompts sent from the Cod